Listen

Lab 10: Recursion: Count Paths and Newton's Method

50 points · Unit 5 · Due Sunday, October 25, 11:59 PM

> This lab has a **Mini Oral Inquiry** (Mini Oral 4).

Two recursions with different shapes, because recursion is not one idea.

Part A is tree recursion. Each call spawns two more, the work explodes exponentially, and the same subproblems get solved over and over. Memoization fixes it, and seeing the fix work is the best possible preparation for Floyd's algorithm in Unit 8.

Part B is linear recursion, where each call spawns one. The interesting part is the base case: it is not an exact match but a *tolerance*, because you are converging on an irrational number and will never land on it exactly.

Specification

Part A — Count Paths

Given an N×N grid, find the total number of paths from the lower left cell (1,1) to the upper right cell (N,N). Only upward and rightward movements are allowed.

This recursive function counts the total number of paths:

```
int NumPaths(int row, int col, int n){
      if(row == n || col == n){
          return 1;
      }
      else{
          return NumPaths(row + 1, col, n)
               + NumPaths(row, col + 1, n);
      }
  }
```

The initial call for a 5×5 grid is `NumPaths(1, 1, 5)`.

Required work:

1. Trace, by hand, the 20 paths found for a 4×4 grid. Verify by executing `NumPaths(1, 1, 4)`.

2. Using **memoization**, improve the efficiency of the function by keeping intermediate values in an N×N array of integers, eliminating the redundant recalculation of paths already found. Test and confirm your memoized version produces identical results to the original.
3. Compare the efficiency of both working versions, in **time and space**.

Part B — Square Root by Newton's Method

Use Newton's Method to find the square root of a nonnegative real number *x* within a tolerance ε.

1. Set `a = x`
2. If `|a² − x| ≤ ε`, stop.
3. Replace `a` with `a − (a² − x) / (2a)` and repeat step 2.

Implement and test **both an iterative and a recursive** version.

Step 3 is often written in the equivalent simplified form `a = (a + x/a) / 2`. Either is acceptable; they are algebraically identical.

Files Provided

> **Download `Lab 10 Files`** — it is the item directly below this document in Unit 5. Everything listed here is in that one zip. Unzip it into a single folder and compile there.

- `NumPaths function above`
- `driver of your own design for both parts`

Creative Final Feature (20 points)

This is not extra credit and it is not optional. **Twenty of this lab's 50 points** come from adding something the specification did not ask for. Eight of the fifteen labs carry a Creative Final Feature — one per unit — and this is one of them.

Extend one part. Directions for Part A: a version that returns the paths themselves rather than the count, an iterative dynamic programming version filling the table bottom-up, or instrumentation counting calls with and without memoization. For Part B: a version handling negative input gracefully, convergence-step counting, or a comparison against `std::sqrt` across a range of inputs.

> **Feature Documentation** — answer all four, in a comment block at the top of your main file:
>
> 1. What does your feature do?
> 2. What concept from your AI Learning Query did it use?

> 3. What did you try that did not work?
> 4. What would you do next if you had another week?

AI Learning Query

Before you write your Creative Final Feature, have this conversation. Start with the prompt below, continue for at least five exchanges, paste your own code in at least once, and push back at least once when an answer seems wrong or incomplete. **This is the graded transcript for Unit 5** — submit it to the AI Learning Transcript 5 dropbox.

> *"I have a recursive NumPaths function in C++ that recomputes the same subproblems many times, and I want to add memoization. Do not write the memoized version for me. Help me understand what exactly gets stored, when I check the table versus when I write to it, and how to tell an uncomputed cell from a legitimately computed zero. Then tell me what memoization costs me in space and whether that trade is always worth making."*

STL Counterpart

> **Ungraded — read this, do not submit it.** Nothing in the STL counts lattice paths, but `std::unordered_map` is how memoization is usually written when the key space is sparse rather than a dense N×N grid. Worth a look to see when a hash table beats an array as a memo table — a question that returns in Unit 7. `std::sqrt` exists, obviously; the point of Part B is the convergence, not the answer.

Be Ready to Answer

These are the questions for **Mini Oral Inquiry 4**. You will record yourself answering them with your code on screen. Question 1 is also submitted in writing as part of this lab.

Every question below is about *your* submission. You may use AI to prepare — that is encouraged — but the answers must describe the code you actually wrote.

1. State the invariant your memoized `NumPaths` maintains about the memo table — what is true of a cell once it has been written? Write this as a comment.
2. Report the actual call counts for `NumPaths(1,1,5)` with and without memoization from *your* instrumented code. Then explain the difference in terms of what the recursion tree looks like in each case.

3. Your hand trace found 20 paths on a 4×4 grid. Show it, and show your program's output agreeing.
4. In Part B, what value of ε did you use, and what happens to your recursion if you make it dramatically smaller — say 1e-18? Try it and report what you observed.
5. Compare your recursive and iterative Newton implementations. Which uses more stack, by how much roughly, and did the compiler appear to optimize the recursion away?

Grading

Criterion	What earns the points	Pts
Correctness and Specification	Every requirement in the specification is met and demonstrated by the driver. Output matches what is stated above. Edge cases named in the specification behave correctly.	**15**
Data Structure Implementation	The required structure is implemented as specified. No STL container is substituted for work the specification asks you to do yourself. Memory is allocated and released correctly.	**8**
Design and Documentation	Interface separated from implementation. Preconditions and postconditions documented on every public function you write or change.	**4**
Coding Conventions	Follows the Oakton C++ Coding Conventions: naming, indentation, commenting, file organization.	**3**
Required work subtotal		**30**
CFF — Feature implemented and runs without errors		**6**
CFF — Feature is genuinely creative and self-chosen		**4**
CFF — Feature demonstrates the concept from your AI Learning Query		**6**
CFF — Feature Documentation complete (all		**4**

four questions)	
Total	**50**

Submitting. Upload all source files plus your driver to the Lab 10 dropbox. **Late submissions are not accepted** — the dropbox closes at the due date. Office hours are virtual, 30-minute slots, Monday through Friday 7:00–8:00 PM; the booking link is in *Start Here*.